

Faculty of Information Technology and Artificial Intelligence
Department of Computer Science

Natural language processing Report

IMDb

Prepared by:

Shahd Batta

Raneem Amer

Supervisor:

Dr .Ahmed Fahil

1. Introduction and Problem Statement

Every day, thousands of people post movie reviews on websites such as IMDb. Reading all of them by hand to find out whether audiences liked a film is slow, and at this scale it is practically impossible. Sentiment analysis solves this by deciding automatically whether a text expresses a positive or a negative opinion. Film studios and streaming platforms can use it to track how audiences react to a release, and review sites can use it to summarize thousands of opinions in a single score. The same technique also works for product reviews and customer feedback.

In this project the task is binary text classification. The input is the text of one movie review, and the output is a label: Positive (1) or Negative (0). This is an NLP problem because the label depends on how the writer uses language. Word choice matters (“brilliant” against “boring”), and so do short phrases (“waste of time”) and negation (“not good”). Single words can also mislead. In our training data, one negative review calls the film “a good film” and says “I do commend the filmmakers”, while a positive review describes its film as “grim”, “frightening” and “disjointed”. Sarcasm and mixed opinions make the task even harder.

We built a classical NLP pipeline for this task: text preprocessing, corpus statistics, N-gram analysis, a trigram language model, a Bag of Words representation, and two classifiers (Multinomial Naive Bayes and Logistic Regression). The best model, Logistic Regression with unigram and bigram features, reached an accuracy of 88.83% on 25,000 unseen test reviews.

2. Dataset and Train/Test Separation

We used the Stanford Large Movie Review Dataset _[1], which we loaded from Hugging Face (stanfordnlp/imdb) [2]. It contains 50,000 English movie reviews from IMDb. A review counts as positive if the writer rated the film 7/10 or higher and as negative if the rating was 4/10 or lower, so neutral reviews are not included. The dataset allows at most 30 reviews per movie, and the training and test sets contain different movies. However, 123 review texts appear in both splits. These are duplicates in the original dataset; they make up less than 0.5% of the test set, and we kept the official split unchanged so that our results can be compared with other work on IMDb.

Split	Total	Positive	Negative
Train	25,000	12,500	12,500
Test	25,000	12,500	12,500

Table 1 Size and class distribution of the dataset

Both splits are perfectly balanced, so the model has no reason to prefer one class, and accuracy is a fair metric. Review length varies a lot. In the training set the average review has 234 words and the median is 174, while the shortest review has 10 words and the longest has 2,470. When we read the raw text, we found HTML line breaks (
) in many reviews. They were left over from the website and carry no opinion, so the preprocessing step removes them.

We used the official train/test split without changes. The test set was saved to data/imdb_test.csv with an id column for each review (test_0, test_1, ...), so the evaluation can be reproduced. We did not use it to build the vocabulary, choose settings or train any model; it was used only once, for the final evaluation. To compare models and settings, we held out 10% of the training data (2,500 reviews) as a validation set, using a stratified split with random_state=42. After choosing the best configuration, we retrained it on all 25,000 training reviews.

The original data is sorted, with all negative reviews before the positive ones. Taking the first reviews for any experiment would therefore give only negative examples. For this reason, the 10,000 reviews used to train the language model were drawn as a random sample (random_state=42), which gave 5,038 negative and 4,962 positive reviews.

3. Text Preprocessing

We wrote one preprocessing function, preprocess(), and applied it to the training reviews, the test reviews, and any new text typed into the demo. All three must go through the same steps, otherwise the classifier would see words in a different form from the one it learned. Table 2 lists the steps in the order they run.

Step	Operation	Example	Reason
1	Remove HTML tags	 → space	Leftover markup from the website
2	Convert ratings	2/10 → ratingbad	Keeps the score before digits are removed
3	Lowercase	NOT → not	Good and good should be one word
4	Expand contractions	didn't → did not	Keeps not as a separate token
5	Remove URLs, digits and punctuation	www.imdb.com, 1967, ! → removed	They rarely carry an opinion
6	Tokenize	text → list of words	Splits the text into units we can count
7	Remove stop words, keep negations	the, was removed; not kept	Frequent words with little meaning
8	Remove one-letter tokens	u s (from U.S.) removed	Fragments left by punctuation removal
9	Lemmatize	movies → movie, loved → love	Groups forms of the same word

Table 2 Preprocessing steps

Two of these steps are our own design decisions. The first concerns negation. The NLTK English stop-word list [3] contains not, no and never. Removing them would turn “not good” into “good” and reverse the meaning of the review, so we removed the negation words (not, no, nor, never, none, nothing, nobody, neither, without) from the stop-word list before using it. Expanding contractions in step 4 supports this choice: without it, “didn’t” would become “didn” and “t” after punctuation removal, and the not would be lost.

The second decision concerns ratings. Many reviewers write a score such as “2/10” in their text, which is a strong sign of their opinion. Removing all digits would delete it, but keeping every number would add noise such as years (“1967”) and durations (“2 hours”). Before removing digits, we therefore convert any score written as X/10 into a word: scores from 0 to 4 become ratingbad, 5 and 6 become ratingmid, and 7 to 10 become ratinggood. Both decisions proved useful later: ratinggood and ratingbad became the strongest features of the final classifier, and the bigram not disappoint received one of the highest positive weights (Section 8).

Original text	After preprocessing
I didn’t like this movie! The acting was NOT good... 2/10	not like movie act not good ratingbad
Absolutely loved it. The actors were amazing. 9/10!	absolutely love actor amaze ratinggood
It was released in 1967 and runs 2 hours.	release run hour

Table 3 Examples before and after preprocessing

For word normalization we compared the Porter stemmer with the WordNet lemmatizer (Table 4). Stemming cuts word endings by fixed rules and often produces non-words such as movi and terribl. Lemmatization returns the dictionary form, which keeps the N-gram tables in Section 5 readable, so we chose it. Our lemmatizer first treats each word as a verb and then as a noun. This simple approach has a limitation: adjectives are not always reduced. For example, better stays better (with the adjective tag it would become good), and amazing becomes the verb amaze. A more accurate solution would use part-of-speech tagging, at a higher processing cost.

Word	Porter stem	WordNet lemma
movies	movi	movie
loved	love	love
studies	studi	study
terrible	terribl	terrible
better	better	better

Table 4 Stemming compared with lemmatization

Preprocessing all 50,000 reviews took about 19 seconds. Applying the function to the test set does not leak information, because it is a fixed set of rules and learns nothing from the data. The language model in Section 6 uses a lighter version of this preprocessing, described there.

4. Corpus Statistics

All statistics in this section come from the 25,000 training reviews only. Table 5 compares the corpus before stop-word removal (after cleaning and tokenization only) with the corpus after the full preprocessing.

Statistic	Before stop-word removal	After preprocessing
Total tokens	5,922,808	3,061,631
Vocabulary size (unique words)	73,192	57,447
Average tokens per review	236.9	122.5
Words appearing only once (hapax)		22,420 (39.0%)

Table 5 Corpus statistics of the training set

Preprocessing removed 48% of the tokens but only 21% of the vocabulary. Stop words are few in number but extremely frequent; the word the alone appears about 336,000 times. Removing them therefore shortens the reviews a lot while hardly changing the number of distinct words. Most of the reduction in vocabulary comes from lemmatization, which merges forms such as movie and movies, or love, loved and loving, into one entry.

The vocabulary also has a long tail. 22,420 words, or 39% of the vocabulary, appear only once in the whole training set. Most of them are names of actors and characters, rare words or spelling mistakes. This pattern is typical of natural language and agrees with Zipf's law [4], where a small number of words account for most of the text and most words are rare. Such words cannot help the classifier, so the Bag of Words step ignores features that appear in fewer than three reviews (Section 7).

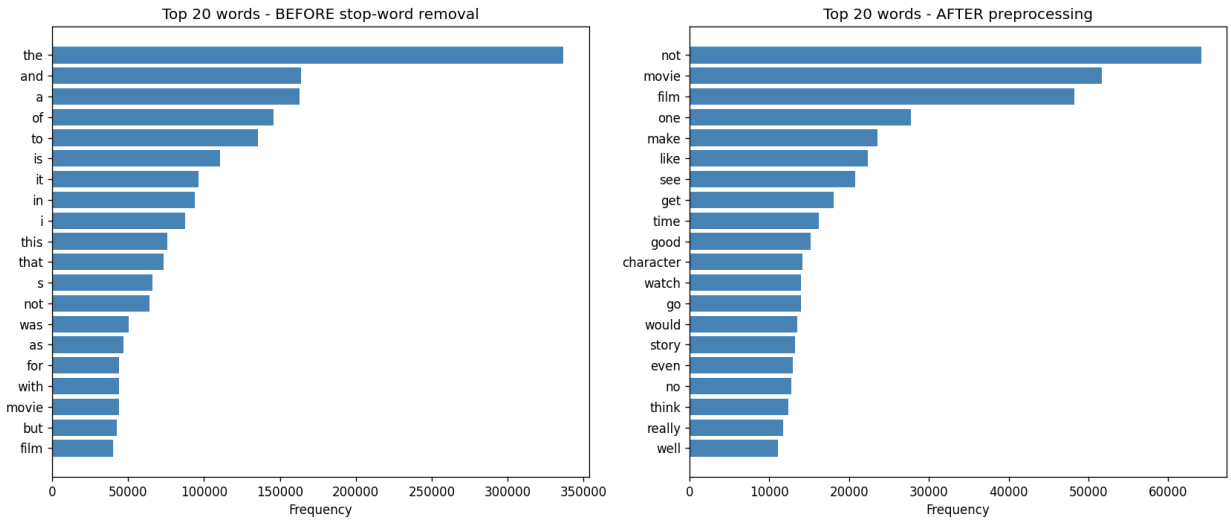


Figure 1 The 20 most frequent words before stop-word removal (left) and after preprocessing (right)

Figure 1 shows how preprocessing changes the picture of the corpus. Before stop-word removal, all 20 top words are function words such as the, and, a and of, and they reveal nothing about the topic or the opinions. After preprocessing, the list is clearly about films: movie, film, character, story and watch. The most frequent token becomes not, with about 64,000 occurrences, which is a direct result of expanding contractions and keeping negation words. Words like movie, film and one are frequent, but they describe the topic and appear in both classes. None of them is among the characteristic words of either class in Section 5, so frequency alone does not tell us which words carry sentiment.

5. N-gram Analysis

An N-gram is a sequence of N consecutive tokens: a unigram is one word, a bigram two and a trigram three [5]. We extracted N-grams from the preprocessed training reviews, so stop words are removed but negation words are kept. Table 6 lists the most frequent N-grams in the whole training set.

#	Unigram	Bigram	Trigram
1	not (64,113)	movie not (2,500)	movie ever see (447)
2	movie (51,713)	not even (2,247)	worst movie ever (386)
3	film (48,209)	could not (1,993)	not waste time (304)
4	one (27,748)	film not (1,831)	film ever see (230)
5	make (23,558)	look like (1,807)	movie ever make (209)
6	like (22,371)	not know (1,751)	make no sense (185)
7	see (20,799)	watch movie (1,553)	worst film ever (174)
8	get (18,017)	would not (1,505)	one worst movie (167)
9	time (16,175)	ever see (1,464)	not get wrong (165)
10	good (15,204)	not see (1,423)	film ever make (166)

Table 6 The 10 most frequent unigrams, bigrams and trigrams (count in parentheses)

Negation dominates the bigrams: seven of the ten most frequent bigrams contain not. Unigrams mostly name the topic (movie, film, character), while trigrams are already full evaluative phrases, and most of the top ten are negative (“worst movie ever”, “not waste time”, “make no sense”). The most frequent trigram, “movie ever see”, appears near the top for both classes (Figure 2). It comes from “the best movie I have ever seen” as well as “the worst movie I have ever seen”, and the word that decides the meaning falls just outside the three-word window. Some frequent phrases also describe the story of a film instead of the writer’s opinion, such as “new york city” and “world war ii” among positive reviews.

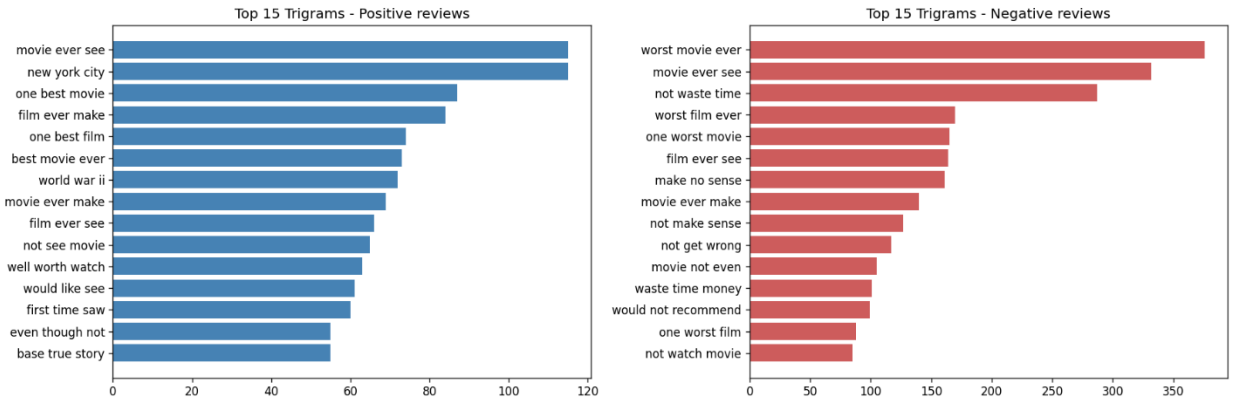


Figure 2 The 15 most frequent trigrams in positive (left) and negative (right) reviews

Raw frequency shows what is common, but a phrase can be common in both classes. To find phrases typical of one class, we scored each N-gram g with a smoothed log ratio:

score(g) = log(P(g | positive) / P(g | negative))

Each probability is the N-gram's count in that class, plus one, divided by the total number of N-grams in the class. Adding one avoids division by zero for phrases that never appear in one of the classes. A score of +2.86 means the phrase is about e^2.86 ≈ 17 times more frequent in positive reviews, a negative score means the opposite, and a score near zero means the phrase is neutral.

Our first attempt kept every N-gram that appeared at least 30 times, and both lists filled with names of films, actors and characters (edie, antwone, gunga din, boll). The cause lies in the dataset: it contains up to 30 reviews of the same movie, so the title of a well-liked film can appear 50 or 60 times, all in positive reviews, without saying anything about sentiment in general. We fixed this by counting document frequency, the number of different reviews a phrase appears in, and keeping only unigrams and bigrams that appear in at least 100 reviews. A single film cannot reach that number, while genuine opinion words appear in thousands of reviews. Trigrams are rarer by nature and showed few names, so for them we kept a threshold of 30 reviews. Table 7 shows the results.

	Most positive	Most negative
Unigrams	ratinggood (+2.86), flawless (+2.43), superbly (+2.24), perfection (+2.13), wonderfully (+1.99)	ratingbad (−3.86), unwatchable (−3.07), mst (−2.88), incoherent (−2.84), stinker (−2.80)
Bigrams	well worth (+2.95), definitely worth (+2.79), first rate (+2.46), highly recommend (+2.41), favorite movie (+2.13)	worst film (−3.86), piece crap (−3.57), worst movie (−3.50), one worst (−3.44), avoid cost (−3.33)
Trigrams	well worth watch (+3.43), well worth see (+2.68), film one best (+2.68), one best movie (+2.65), one favorite movie (+2.56)	not waste money (−4.21), worst movie see (−4.13), one worst movie (−4.05), not even worth (−4.04), one worst film (−3.83)

Table 7 The most characteristic N-grams of each class (log-ratio score)

The rating tokens from our preprocessing are the most characteristic unigrams of both classes, and ratingbad is about 47 times more frequent in negative reviews. The other top unigrams are strong, fairly rare words such as flawless and unwatchable. Common words like good and bad are missing from the lists, because they also appear often in the opposite class (“not good”, “good actors but...”). A few names remain after the fix, and they carry a real signal. powell (the actor William Powell) and the trigram “daniel day lewis” refer to actors whose many films are well reviewed. mst comes from Mystery Science Theater 3000, a comedy show that mocks bad films, so comparing a film to it is a criticism. Some entries reflect how reviewers write: underrate (“this film is underrated”) and “pleasantly surprise” are positive, and sadness leans positive, probably because sad dramas are often praised.

Negation also behaves in more than one way. “not even worth” and “not worth time” are negative, as expected. In “not waste money”, which comes from “don’t waste your money”, the negation of a negative word still gives a negative phrase. A rule that simply flips the sentiment of any word after not would get this wrong, which is one reason to let the classifier learn bigram weights from the data.

6. N-gram Language Model

A language model assigns a probability to the next word given the words before it. We built a trigram model, which estimates this probability from the two previous words using counts from the training data [5]:

P(w3 | w1 w2) = C(w1 w2 w3) / C(w1 w2)

For example, the probability that “recommend” follows “i highly” is the number of times “i highly recommend” appears, divided by the number of times “i highly” appears. Many two-word contexts never occur in the training data, so the model uses a simple backoff: if the trigram context is unknown, it uses the bigram probability P(w3 | w2), and if the last word is also unknown, it falls back to unigram frequencies.

The language model needs real word sequences, so it uses lighter preprocessing than the classifier. We kept stop words and did not lemmatize, since words like “the”, “of” and “is” are part of natural sentences and are often the correct next word. Each review was lowercased, cleaned of HTML tags, had its contractions expanded, and was split into sentences at “.”, “!” and “?”. Each sentence was then wrapped in start markers (<s> <s>) and an end marker (</s>), so the model also learns how sentences begin and end. We trained the model on a random sample of 10,000 training reviews (5,038 negative, 4,962 positive). It contains 50,586 distinct words and 644,734 distinct two-word contexts. This large number shows the data sparsity problem of N-gram models: as N grows, the number of possible sequences explodes, and most of them are seen only once or twice.

Prompt	Top 5 next words
this movie is	a (0.10), not (0.07), the (0.04), that (0.04), about (0.03)
one of the best	of (0.08), part (0.04), thing (0.04), movies (0.03), </s> (0.03)
the acting was	so (0.07), terrible (0.06), not (0.06), good (0.05), horrible (0.04)
i would not	be (0.16), have (0.15), recommend (0.08), even (0.05), want (0.03)
a complete waste of	time (0.51), money (0.07), film (0.05), your (0.05), a (0.04)
i highly	recommend (0.83), doubt (0.04), recomend (0.02), urge (0.02), recommended (0.02)
the special effects	are (0.16), </s> (0.14), and (0.10), were (0.08), that (0.02)
worst movie i have	seen (0.17), ever (0.16), to (0.14), not (0.10), never (0.04)

Table 8 Next-word predictions of the trigram model (top 5 with probabilities). All prompts used the trigram level

For frequent contexts the predictions are sensible. After “i highly” the model is almost certain of “recommend” (0.83), and it even learned the misspelling “recomend” from the data. “a complete waste of” is followed by “time” in half of the cases. The clearest weakness appears with “one of the best”. The model predicts “of” instead of “movies”, because it only sees the last two words, “the best”, which are often followed by “of” (“the best of the year”). The words “one of” fall outside its window. For “the acting was” the predictions are mixed, and negative words have higher probabilities (terrible 0.06 and horrible 0.04, against good 0.05), which suggests that reviewers criticize acting more often than they praise it.

Backoff is visible with unusual prompts. “the purple elephant” never occurs in the data, so the model backed off to the bigram level and predicted from “elephant” alone (“walk”, “man”, “stampede”; “man” probably comes from the film *The Elephant Man*). For the invented word “xyzabc” it backed off to unigrams and returned the most common words (“the”, </s>, “and”).

We also generated text greedily by repeatedly appending the most probable next word:

- “this movie is” → “this movie is a very good”
- “the plot” → “the plot is not a bad movie”
- “the acting was” → “the acting was so bad it s a good movie”

Every three-word window in these sentences is grammatical, but the sentences as a whole can contradict themselves, as in the last example, because the model forgets “bad” once it leaves the window. Greedy search also drifts toward the same common phrases (“a good movie”). (“it s” comes from “it’s”: our language-model preprocessing expanded only “n’t” contractions.)

To see whether the two classes use different language, we trained one model on the positive reviews of the sample and one on the negative reviews (Table 9).

Prompt	Positive model	Negative model
the acting was	good, great, superb, not, outstanding	terrible, so, not, horrible, good
i would	have, not, recommend, like, say	not, have, like, say, be
the ending	is, </s>, was, of, which	is, </s>, was, of, which

Table 9 Top 5 predictions of separate positive and negative language models

For evaluative contexts the two models differ clearly: “the acting was” continues with “superb” and “outstanding” in positive reviews and with “terrible” and “horrible” in negative ones. For “the ending” the predictions are identical, because the next words are linking words and the opinion comes later in the sentence. Sentiment is carried by specific words, and this is what the Bag of Words classifier in Section 7 relies on.

Finally, we measured perplexity on 500 random reviews from the test set. Perplexity shows how surprised a model is by unseen text, and lower values are better [5]. A perplexity of 100 roughly means the model is as uncertain as if it chose among 100 equally likely words at each step. For this measurement we used a bigram model with add-k smoothing, which adds k to every count so that no unseen word pair gets a probability of zero. This was a measurement only; no setting of the classifier was chosen from it.

k	Perplexity
1 (Laplace)	2355.2
0.1	892.9
0.01	558.6

Table 10 Bigram perplexity on 500 unseen test reviews for different smoothing values

Add-1 smoothing gives a very high perplexity. With a vocabulary of about 50,000 words, adding one to every possible word pair moves too much probability from pairs that really occur to pairs that never occur. Lowering k to 0.01 reduced perplexity by a factor of four. Better smoothing methods such as Kneser-Ney, or the neural language models covered in Project 2, would reduce it further.

7. Bag of Words Representation and Classification

7.1 Bag of Words

A classifier works with numbers, so each review must become a vector. The Bag of Words (BoW) representation counts how many times each word appears in a review and ignores word order. We used scikit-learn's CountVectorizer [6]. Each row of the resulting matrix is a review, each column is a feature from the vocabulary, and each cell holds a count. In a small example with the sentences "the movie was good" and "the movie was not good", the two vectors differ only in the column for not. With ngram_range=(1, 2) the vectorizer also adds bigrams as features, so "not good" gets its own column and the classifier can learn a weight for the phrase itself. This brings back a small part of the word order that BoW otherwise loses.

The vectorizer learns its vocabulary from the training data only (fit_transform). The validation and test reviews are converted with the same vocabulary (transform), and any word that never appeared in training is ignored. Fitting the vocabulary on the test set would leak information from it, while ignoring unseen words matches how the model behaves on new reviews after deployment. We used two settings to reduce noise. min_df=3 drops features that appear in fewer than three reviews, which removes most of the hapax words from Section 4, and max_df=0.95 drops features that appear in more than 95% of reviews.

On 22,500 training reviews, unigrams alone gave 25,301 features, and unigrams with bigrams gave 167,978. The matrix is very sparse: only 3,369,379 of its 3.78 billion cells (0.089%) are non-zero, which means a review uses about 150 of the 167,978 features on average. Scikit-learn stores only the non-zero cells, so the large vocabulary does not cause memory problems.

7.2 Classifiers

We compared two classical classifiers.

Multinomial Naive Bayes estimates, for each feature, how likely it is to appear in positive and in negative reviews. For a new review it multiplies these probabilities with the prior probability of each class and picks the class with the higher result. It assumes that features are independent given the class, which is not true for language, but the model is fast and often works well on word counts [5]. We used alpha=1.0, which is the same add-one smoothing used in Section 6: a feature never seen in one class does not get a probability of zero.

Logistic Regression learns one weight per feature. A review's score is the sum of each feature count multiplied by its weight, and the sigmoid function turns that score into a probability of being positive [5]. Strongly positive words get large positive weights, neutral words get weights near zero, and all weights are learned together, so the model can adjust for features that often appear together. We used C=0.5, a regularization setting that keeps the weights of rare features from

growing too large and helps the model generalize, with the liblinear solver and up to 2,000 iterations.

7.3 Model selection on the validation set

We trained both classifiers with both feature settings on the 22,500 training reviews and evaluated them on the 2,500 validation reviews. Each run used its own vectorizer, fitted on the training part only. The test set was not used at this stage.

Model	Features	Vocabulary size	Accuracy	F1
Logistic Regression	unigrams + bigrams	167,978	0.8912	0.8926
Naive Bayes	unigrams + bigrams	167,978	0.8788	0.8770
Logistic Regression	unigrams	25,301	0.8724	0.8731
Naive Bayes	unigrams	25,301	0.8580	0.8541

Table 11 Validation results (2,500 reviews), sorted by F1

Adding bigrams increased the vocabulary 6.6 times and improved accuracy by about two points for both models (+2.08 for Naive Bayes and +1.88 for Logistic Regression). For Logistic Regression the error rate fell from 12.76% to 10.88%, so about 15% of its errors disappeared. The cost was small: one Logistic Regression model trained in about 16 seconds, and all four experiments took under a minute. The bigrams that helped are the phrases found in Section 5, such as “not good”, “waste time” and “well worth”. Logistic Regression was better than Naive Bayes in both settings, by 1.2 to 1.4 points of accuracy, which fits the difference described above: Naive Bayes judges each feature on its own, while Logistic Regression learns all weights together.

We selected Logistic Regression with unigram and bigram features. Since the validation set had done its job, we added it back to the training data and retrained the selected model on all 25,000 training reviews. With more data, more phrases passed the min_df=3 threshold, and the final vocabulary grew to 186,068 features. The complete procedure was:

1. Split the 25,000 training reviews into a training part (22,500) and a validation set (2,500).
2. Train four configurations on the training part and compare them on the validation set.
3. Retrain the best configuration on all 25,000 training reviews.
4. Evaluate it once on the 25,000 test reviews (Section 8).

8. Evaluation and Error Analysis

8.1 Results on the test set

The selected model was evaluated once on the 25,000 reviews of the official test set. The test reviews went through the same preprocess() function and were converted with the vectorizer fitted on the training data. We treat Positive as the positive class when computing precision and recall. Precision is the share of reviews predicted Positive that really are positive, recall is the share of truly positive reviews that the model found, and F1 is the harmonic mean of the two.

Accuracy	Precision	Recall	F1
0.8883	0.8913	0.8846	0.8879

Table 12 Test results of the selected model (Logistic Regression, unigrams + bigrams)

Class	Precision	Recall	F1	Support
Negative	0.8854	0.8921	0.8887	12,500
Positive	0.8913	0.8846	0.8879	12,500

Table 13 Results per class on the test set

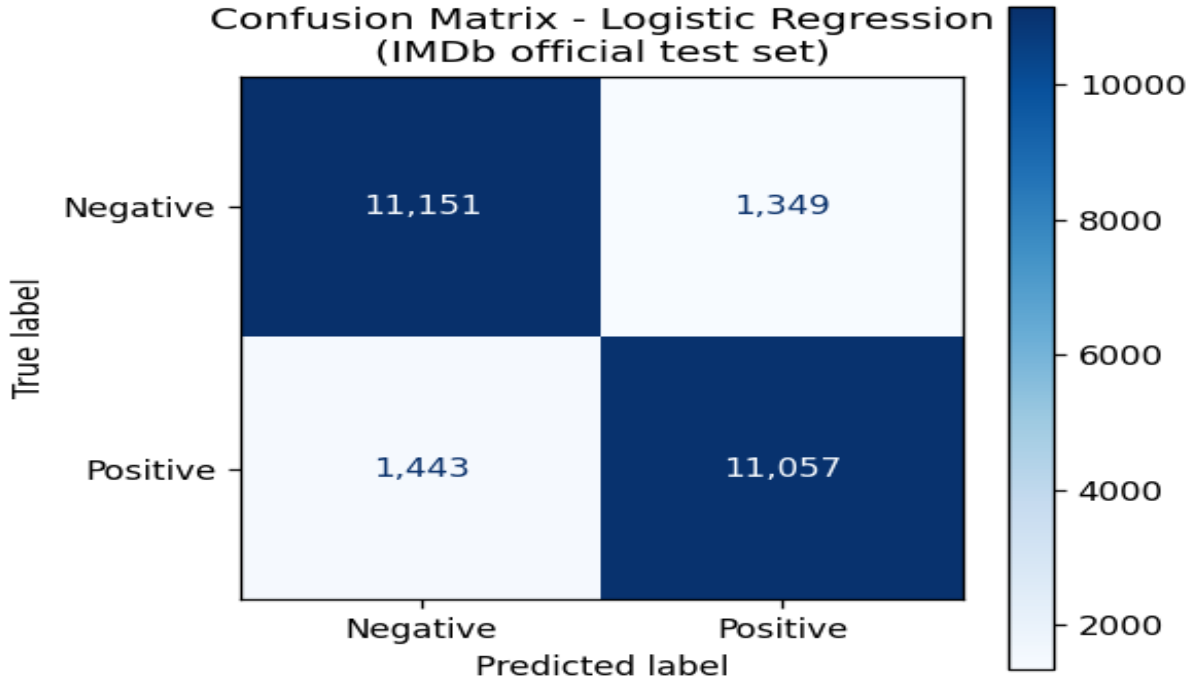


Figure 3 Confusion matrix on the test set

The model classified 22,208 of the 25,000 test reviews correctly and made 2,792 errors: 1,349 false positives (negative reviews predicted Positive) and 1,443 false negatives (positive reviews predicted Negative). The two error types are almost equal, and the F1 scores of the two classes differ by less than 0.001, so the model is balanced and does not favor either class. Test accuracy (88.83%) is very close to validation accuracy (89.12%). The model generalizes well to unseen reviews, and the validation set gave a reliable estimate of its real performance. A large gap between the two would have pointed to overfitting or to data leakage.

For comparison, we also evaluated Naive Bayes, trained on the same features, on the test set (Table 14). This comparison is only reported here; the model choice was made earlier on the validation set.

Model	Accuracy	Precision	Recall	F1
Naive Bayes	0.8564	0.8841	0.8203	0.8510
Logistic Regression	0.8883	0.8913	0.8846	0.8879

Table 14 Both classifiers on the test set

Naive Bayes has a similar precision but a much lower recall (0.8203): it misses 18% of the positive reviews, so it predicts Negative too often. A likely reason is that negative reviews contain many

strong, frequent words such as worst, waste and awful, and Naive Bayes adds the effect of each word separately, so a few negative words can outweigh the rest of a positive review. Logistic Regression learns all weights together and gives more balanced results.

8.2 Individual predictions

Table 15 shows 12 test reviews chosen at random (random_state=42), with the true label, the predicted label and the model's probability that the review is positive. The predictions for all 25,000 test reviews are saved in results/test_predictions.csv.

ID	Review (beginning)	True	Predicted	P(positive)
test_6868	I could not believe how terrible and boring this Hollywood remake was...	Negative	Negative	0.001
test_24016	I rented Boogie Nights last week and... I had a blast...	Positive	Positive	0.983
test_9668	First off, this movie is not near complete...	Negative	Negative	0.001
test_13640	I watched this mini in the early eighties. Sam Waterson proved himself to be a great actor...	Positive	Positive	0.949
test_14018	This movie was never intended as a big-budget film but was a cute little picture...	Positive	Positive	0.827
test_7488	I thought this was an extremely bad movie...	Negative	Negative	0.266
test_5804	This was one of the biggest pieces of crap I have ever had to watch...	Negative	Negative	0.000
test_12909	I just watched it for the second time today and I must say with all my heart...	Positive	Positive	0.943
test_3386	I've read through a lot of the comments here about how this movie sticks to the book...	Negative	Positive ✗	0.778
test_9567	I usually much prefer French movies over American ones... this movie was very disappointing...	Negative	Negative	0.072
test_21423	This was very funny, even if it fell apart a little at the end...	Positive	Negative ✗	0.024

ID	Review (beginning)	True	Predicted	P(positive)
test_3503	SPOILER!!!! Mind Ripper hmmmm....	Negative	Negative	0.002

Table 15 Individual predictions on random test reviews

Ten of the twelve predictions are correct, close to the overall accuracy. Most correct predictions are made with high confidence, while test_7488 is correct but uncertain (0.266) even though it says “extremely bad movie”. In test_3386 the writer discusses the book the film is based on, and the model cannot tell whether the praise is for the book or the film. Test_21423 is discussed in Section 8.4.

8.3 What the model learned

The weights of the Logistic Regression model show which features push a review toward each class (Table 16).

#	Positive	Negative
1	ratinggood (+2.09)	ratingbad (−2.36)
2	not disappoint (+1.34)	worst (−1.62)
3	excellent (+1.15)	not recommend (−1.42)
4	perfect (+1.04)	awful (−1.39)
5	superb (+1.04)	disappointment (−1.34)
6	refresh (+0.94)	poorly (−1.18)
7	funniest (+0.91)	not worth (−1.13)
8	wonderful (+0.89)	bore (−1.13)
9	must see (+0.89)	waste (−1.08)
10	wonderfully (+0.86)	dull (−1.06)

Table 16 Features with the strongest weights

The two rating tokens are the strongest features of the model, which confirms the value of converting written scores during preprocessing. Bigrams with negation are among the strongest features in both directions. The word disappoint alone has a negative weight (−0.98), but the bigram not disappoint has the second highest positive weight (+1.34), and not recommend, not worth and not funny are strongly negative. None of these features would exist if not had been removed as a stop word. Most of the other strong features match the characteristic N-grams of Section 5. A few have

a clear explanation from typical phrases: rare is positive (“a rare gem”), today is positive (“still holds up today”), and save is negative (“save your money”).

8.4 Error analysis

Errors are slightly more frequent in long reviews. Accuracy is 89.67% for the shortest quarter of the test reviews and 88.09% for the longest quarter. Long reviews often contain a plot summary and mixed opinions. The model’s probabilities are also informative: its average confidence is 0.937 when it is correct and 0.775 when it is wrong, so low-confidence predictions are more likely to be errors. Some errors, however, were made with a confidence close to 1.0.

Test_21423 is a positive review that the model classified as negative with high confidence (P(positive) = 0.024). After preprocessing, the review contains positive words such as funny (twice), best and lucky, but also fell apart, sad, no friend, frustrate, steal, horrible and kill. Most of the negative words describe the characters and the plot. “Anne Ramsey, who is so horrible that even Mother Theresa would have wanted to kill her” is praise for an actress playing a villain. “Does not go overboard” is also praise, but the model sees not and overboard separately.

We then read the three false positives and three false negatives that the model predicted with the highest confidence (all close to 1.0) and grouped them by the cause of the error (Table 17).

ID	True → Predicted	Cause	Explanation
test_588	Negative → Positive	Plot summary	Most of the text retells the story of the sapphires and “the greatest conqueror”, and the writer’s opinion only begins after a long summary.
test_776	Negative → Positive	“So bad it’s good”	The writer remembers the film fondly: it “scared the hell out of us kids, in a fun way” and has “one of the greatest unintentionally funny” endings.
test_1152	Negative → Positive	Mixed opinion	The writer praises another director, Bergman, at length, and criticizes the film’s director in a subtle, indirect way.
test_13811	Positive → Negative	Contrast	“horrid day” (twice) describes the writer’s day, and “the budget is obviously extremely low... but look what they did with it!” is praise.

ID	True → Predicted	Cause	Explanation
test_13929	Positive → Negative	Plot summary, genre	A horror review full of words like “gore”, “nastiest”, “gruesome” and “hacked to death”, which are praise for fans of the genre.
test_13933	Positive → Negative	Plot summary, genre	A second review of the same horror film, with “horrible vision”, “cannibalism” and “brutally kill”.

Table 17 Causes of the most confident errors

Two of the six errors are reviews of the same horror film. The vocabulary of horror is negative on the surface (death, blood, killing), so the model is biased against positive horror reviews. All six errors share one cause: Bag of Words counts words but ignores word order, context and what each word refers to. It cannot tell whether “horrible” describes the film, a character or the writer’s day, and it cannot follow a contrast introduced by “but”. These are the limitations that the embedding and Transformer models in Project 2 are designed to address.

8.5 Demo

The function predict_sentiment() runs the full pipeline on any new text: preprocess(), then the BoW vectorizer, then the classifier, both loaded from the saved files in models/. Table 18 shows its output for reviews we wrote to test difficult cases.

Review	Prediction (confidence)
An absolute masterpiece. The performances were stunning and I was moved to tears.	Positive (89.2%)
Two hours of my life I will never get back. Terrible script and wooden acting.	Negative (89.7%)
The movie was not good at all, very disappointing.	Negative (81.0%)
I didn’t expect much, but it did not disappoint. One of the best films this year!	Positive (88.7%)
Great cast, beautiful visuals, but the story was boring and way too long.	Negative (53.2%)
Oh wonderful, another sequel nobody asked for. Just brilliant.	Positive (60.7%) ✗

Review	Prediction (confidence)
I’d give it 3/10.	Negative (91.6%)

Table 18 Demo predictions on new reviews

The model handles negation (“not good”, “did not disappoint”) and written ratings correctly. For the mixed review its confidence is close to 50%, which is a reasonable answer to a review that is half positive and half negative. The sarcastic review is classified as positive, because the model sees wonderful and brilliant and cannot detect the ironic tone.

9. Conclusion and Future Work

We built a complete classical NLP pipeline for sentiment analysis of IMDb movie reviews. A single preprocessing function cleans the text, keeps negation words and converts written ratings into tokens, and it is applied in the same way to training data, test data and new input. The corpus statistics and N-gram analysis showed which words and phrases carry sentiment, and a trigram language model with backoff predicted sensible next words for frequent contexts while exposing the short memory and data sparsity of N-gram models. For classification, a Bag of Words representation with unigrams and bigrams and a Logistic Regression classifier reached 88.83% accuracy and an F1 score of 0.8879 on the 25,000 reviews of the official test set. The test set was used only once, and the result was close to the validation estimate.

Two preprocessing decisions had a measurable effect. The rating tokens became the strongest features of the model, and keeping negation words made bigram features such as “not disappoint” and “not recommend” possible. Adding bigrams reduced the validation error rate of Logistic Regression by about 15%.

The remaining errors come from mixed opinions, sarcasm, contrasts across a sentence, plot summaries, and genres such as horror whose vocabulary sounds negative. All of them require an understanding of word order and context that Bag of Words does not have. Smaller limitations are the verb-then-noun lemmatization, which misses some adjectives, and the add-k smoothing of the language model.

In Project 2 we will reuse the same corpus, the same preprocess() function and the same saved test set, and use the results of this project as a baseline. We plan to compare them with word embeddings, neural network classifiers and a fine-tuned Transformer model such as BERT, which take context into account.

References

[1] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng and C. Potts, "Learning word vectors for sentiment analysis," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL)*, Portland, OR, USA, 2011.

[2] Q. Lhoest et al., "Datasets: A community library for natural language processing," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2021.

[3] S. Bird, E. Klein and E. Loper, Natural Language Processing with Python, Sebastopol, CA, USA: O'Reilly Media, 2009.

[4] G. K. Zipf, Human Behavior and the Principle of Least Effort, Cambridge, MA, USA: Addison-Wesley, 1949.

[5] D. Jurafsky and J. H. Martin, "Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models (3rd ed. draft)," Stanford University, 19 August 2026. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research,* vol. 12, pp. 2825-2830, 2011.